Flutter Basics

Beginner-friendly notes with simple Dart and Flutter examples

What you will learn

- What Flutter and Dart are
- How a Flutter app starts
- Widgets: MaterialApp, Scaffold, AppBar, Text, Container, Row and Column
- StatelessWidget vs StatefulWidget
- Buttons, text fields, layout and basic state
- A complete beginner login-screen example
- Practice tasks and a learning checklist

1. What are Flutter and Dart?

Flutter is a UI toolkit used to build apps for Android, iOS, web and desktop. **Dart** is the programming language used to write Flutter apps. Most of the screen is built by combining widgets.

2. Basic Flutter app structure

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text('My First App')),
        body: const Center(
          child: Text('Hello Flutter!'),
        ),
      ),
    );
  }
}
```

Understand it: main() is the entry point. runApp() starts the app. MaterialApp provides Material Design setup. Scaffold gives a page structure. AppBar is the top bar. Center places its child in the middle. Text displays words.

3. Common widgets

Widget	Purpose
MaterialApp	Top-level app configuration and theme
Scaffold	Basic page layout: app bar, body, drawer, etc.
AppBar	Top app bar
Text	Displays text
Container	Sizing, padding, margin, color, decoration
Row	Places children horizontally
Column	Places children vertically
Padding	Adds space inside the given area around a child
SizedBox	Adds fixed space or constrains width/height
Expanded	Uses available space inside a Row or Column
ElevatedButton	Clickable button
TextField	Accepts user input
ListView	Scrollable list of widgets

4. StatelessWidget vs StatefulWidget

StatelessWidget: use when the widget does not manage changing internal state. **StatefulWidget:** use when the UI must update after something changes, such as toggling password visibility or increasing a counter.

```
// Stateless example
class WelcomeText extends StatelessWidget {
  const WelcomeText({super.key});

  @override
  Widget build(BuildContext context) {
    return const Text('Welcome!');
  }
}

// Stateful example: a counter
class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  State<CounterPage> createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int count = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text('Count: $count'),
            ElevatedButton(
              onPressed: () {
                setState(() {
                  count++;
                });
              },
              child: const Text('Increase'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Remember: setState() tells Flutter to rebuild the relevant UI after a state value changes.

5. Layout basics

```
// Horizontal layout
Row(
  children: const [
    Icon(Icons.home),
    SizedBox(width: 8),
    Text('Home'),
  ],
)

// Vertical layout
Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: const [
    Text('Name'),
    SizedBox(height: 8),
    Text('Email'),
  ],
)

// Space around a child
Padding(
```

```
  padding: const EdgeInsets.all(16),
  child: Container(
    width: 200,
    height: 100,
    alignment: Alignment.center,
    decoration: BoxDecoration(
      color: Colors.blue,
      borderRadius: BorderRadius.circular(12),
    ),
    child: const Text('Card'),
  ),
)
```

Row: main axis is horizontal. **Column:** main axis is vertical. For Row and Column, mainAxisAlignment controls spacing along the main axis; crossAxisAlignment controls alignment across the other axis.

6. Buttons and text fields

```
final TextEditingController nameController =
    TextEditingController();

TextField(
  controller: nameController,
  decoration: const InputDecoration(
    labelText: 'Enter your name',
    border: OutlineInputBorder(),
  ),
)

ElevatedButton(
  onPressed: () {
    final name = nameController.text;
    debugPrint('Name: $name');
  },
  child: const Text('Submit'),
)
```

In a StatefulWidget, dispose controllers when no longer needed: nameController.dispose() inside dispose().

7. Complete beginner login screen

Paste this into lib/main.dart in a new Flutter project. It is a UI practice example only; it does not authenticate against a server.

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorSchemeSeed: const Color(0xFF00529F),
        useMaterial3: true,
      ),
      home: const LoginPage(),
    );
  }
}

class LoginPage extends StatefulWidget {
  const LoginPage({super.key});

  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final emailController = TextEditingController();
  final passwordController = TextEditingController();
  bool hidePassword = true;

  @override
  void dispose() {
    emailController.dispose();
    passwordController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: Center(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const Icon(Icons.lock_outline, size: 64),
              const SizedBox(height: 16),
              const Text(
                'Welcome Back',
                textAlign: TextAlign.center,
                style: TextStyle(
                  fontSize: 26,
                  fontWeight: FontWeight.bold,
                ),
              ),
              const SizedBox(height: 24),
              TextField(
                controller: emailController,
                keyboardType: TextInputType.emailAddress,
                decoration: const InputDecoration(
                  labelText: 'Email',
                  border: OutlineInputBorder(),
                  prefixIcon: Icon(Icons.email_outlined),
```

```
              ),
            ),
            const SizedBox(height: 16),
            TextField(
              controller: passwordController,
              obscureText: hidePassword,
              decoration: InputDecoration(
                labelText: 'Password',
                border: const OutlineInputBorder(),
                prefixIcon: const Icon(Icons.lock_outline),
                suffixIcon: IconButton(
                  icon: Icon(
                    hidePassword
                        ? Icons.visibility
                        : Icons.visibility_off,
                  ),
                  onPressed: () {
                    setState(() {
                      hidePassword = !hidePassword;
                    });
                  },
                ),
              ),
            ),
            const SizedBox(height: 24),
            ElevatedButton(
              onPressed: () {
                if (emailController.text.trim().isEmpty ||
                    passwordController.text.isEmpty) {
                  ScaffoldMessenger.of(context).showSnackBar(
                    const SnackBar(
                      content: Text('Please enter email and password'),
                    ),
                  );
                  return;
                }
                ScaffoldMessenger.of(context).showSnackBar(
                  const SnackBar(
                    content: Text('UI practice: fields are filled'),
                  ),
                );
              },
              child: const Text('Login'),
            ),
          ],
        ),
      ),
    ),
  );
 }
}
```

8. Useful Dart basics

Variables: final means assign once; var lets Dart infer a variable's type. Use const for compile-time constants. **Functions:** reusable blocks of code. **Lists:** ordered collections. **if/else:** conditional logic.

```
final String userName = 'Anu';
var age = 26;
const double appPadding = 16;

void greet(String name) {
  print('Hello, $name');
}

final fruits = <String>['Apple', 'Banana'];

if (age >= 18) {
  print('Adult');
} else {
  print('Under 18');
}
```

9. Common beginner errors

• **RenderFlex overflow:** content does not fit. Try SingleChildScrollView, Expanded, Flexible, or adjust constraints appropriately.
• **setState called after dispose:** avoid updating state after the screen is removed.
• **Controller not disposed:** dispose TextEditingController in State.dispose().
• **Missing comma/bracket:** format the file and inspect the line reported by the error.
• **Widgets not updating:** if state changes in a StatefulWidget, check that the update happens inside setState().

10. Practice checklist

■ Create a screen with an AppBar and centered text.

■ Make a profile card using Container, Padding and Column.

■ Add two TextFields and a button.

■ Create a counter using StatefulWidget and setState().

■ Add a password show/hide icon.

■ Build a simple list with ListView.builder.

Suggested learning order: Dart variables and functions → widgets → layout → input and buttons → state → navigation → API calls → MVVM and project structure.